EF Core 01 — Lab Assignment Report

Mapping Classes to Database using Entity Framework Core (Code First)

Target Framework: .NET 5.0 (C# 9.0)
ORM Engine: EF Core 5.0.17 (SQL Server)
Architecture: Configuration Class Per Entity

1 Why do we use Code First rather than Database First?

CORE PARADIGM

In modern .NET enterprise architecture, **Code First** is the predominant industry standard (used in >95% of new projects). Instead of creating SQL tables in SSMS first and generating C# classes from them, Code First treats the C# domain model as the source of truth for four fundamental reasons:

- **The "C# Only" Dream (Domain-Driven Focus):** Developers write the entire system using pure C# OOP models without context-switching between SQL Server scripts and application code. The business domain drives database design, not physical storage constraints.
- **True Object-Oriented Modeling:** Relational tables cannot natively represent OOP principles. With Code First, we cleanly leverage: **Inheritance** (TPH, TPT), **Aggregation**, and rich relationships via **Navigation Properties** (e.g. `ICollection<Book>`, reference properties) that make querying expressive and type-safe.
- **Multi-DBMS Support via Provider Abstraction:** Because our schema is defined via C# abstractions, switching database engines (e.g., from SQL Server to PostgreSQL, MySQL, or SQLite) requires changing only the NuGet provider package and connection string (`UseSqlServer()` → `UseNpgsql()`). In Database First, changing DBMS requires rewriting all DDL scripts and re-scaffolding models.
- **Maintainability & Migrations as the Single Source of Truth:** EF Core Migrations track every incremental change over time in timestamped C# files containing explicit `Up()` and `Down()` methods. This gives teams full Git version control, seamless CI/CD integration, and automated rollbacks without manual database modifications.

> **Core Rule:** Your C# code is the *single source of truth*. Tables are never modified manually in SQL Server; all changes are applied predictably via Migrations.

2 Why do we use Fluent API over Data Annotations?

CLEAN ARCHITECTURE

While **Data Annotations** (attributes like `[Required]` or `[MaxLength]`) are quick to write, **Fluent API** (configured via `OnModelCreating` and `IEntityTypeConfiguration<T>`) is far superior in production applications:

- **Clean POCO & Separation of Concerns (Single Responsibility):** Domain entities stay "Plain Old CLR Objects" free from database-specific metadata. Placing attributes like `[Table("Writers")]` or `[Column]` inside models pollutes the core business layer with infrastructure concerns. Fluent API decouples persistence configuration entirely.
- **Advanced Configurations Impossible via Data Annotations:** Many database features can *only* be configured through Fluent API: composite primary keys (`HasKey(x => new { x.Id1, x.Id2 })`), precise column precision and scale (`HasColumnType("decimal(8,2)")`), custom identity seed/step (`UseIdentityColumn(10, 10)`), cascade delete behaviors (`OnDelete(DeleteBehavior.Restrict)`), filtered indexes, table splitting, and shadow properties.
- **Precedence Hierarchy:** EF Core resolves configurations in order: `Fluent API (Highest Priority) > Data Annotations > Conventions (Default)` . Fluent API can override any attribute or default convention without modifying the model class.
- **Modularity via `IEntityTypeConfiguration<T>` (Configuration Class Per Entity):** Instead of a bloated `DbContext` with thousands of lines, each entity gets its own dedicated configuration class (e.g., `BookConfigurations.cs`), keeping the codebase maintainable, testable, and modular.
- **Configuring Third-Party / Immutable Classes:** When entity classes are defined in external assemblies or shared libraries where you cannot modify source code to add attributes, Fluent API is the only way to configure their database mappings.

3 Why did the property "Id" become a Primary Key without any explicit configuration?

CONVENTION

This is governed by EF Core's built-in **Convention over Configuration** (Default Behavior):

- **Naming Rule:** EF Core scans public properties of an entity. Any property named either `Id` or `<EntityName>Id` (e.g. `BookId`), case-insensitive, is automatically detected as the table's **Primary Key** without requiring `[Key]` or `HasKey()`.
- **Automatic Identity Generation:** Because `Id` is an integer (`int`), EF Core configures it as a SQL Server `IDENTITY(1, 1)` column by default, meaning values are automatically auto-incremented by the database engine on every insert.

4 Why is "Country" nullable in the database while "Price" is not?

TYPE SYSTEM

This difference is rooted in the **C# Type System** and how EF Core reflects value types vs. reference types into database columns:

Property	C# Type	Type Category	Can Hold Null in C#?	Resulting Database Column
Price	`decimal`	Value Type	**No** (default is `0.0m`)	`decimal(8,2) NOT NULL`
Country	`string`	Reference Type	**Yes** (default is `null`)	`nvarchar(max) NULL`

- **`Price` is NOT NULL:** In .NET, `decimal` is a non-nullable **Value Type**. It must hold a valid numeric value and cannot be null. EF Core reflects this by marking the database column `NOT NULL`. To make it nullable, it must be explicitly defined as `decimal?` (Nullable Value Type).
- **`Country` is NULL:** In C#, `string` is a **Reference Type**. Reference types can naturally hold `null` references in CLR. Therefore, EF Core by convention maps it to a nullable database column (`NULL`). To enforce non-nullability, we must explicitly apply `[Required]` or `.IsRequired()`.

📌 Mapping Approaches Comparison Matrix

Feature / Aspect	1. Mapping by Convention	2. Data Annotations	3. Fluent API (Configuration Class)
Definition Location	Implicit (Default EF rules)	Attributes directly on entity properties	Dedicated `IEntityTypeConfiguration<T>` classes
POCO Cleanliness	100% Clean	Polluted with database concerns	**100% Clean (Complete Separation of Concerns)**
Flexibility & Power	Basic defaults only	Medium (Single property constraints)	**Maximum** (Composite keys, precision, delete behaviors)
Precedence Order	Lowest (3rd)	Medium (2nd)	**Highest (1st) — Overrides all others**
Industry Standard	Prototyping / standard IDs	Simple validations & DTOs	**Enterprise Production Systems**

✅ BookstoreSystem Lab Migrations Applied

1. InitialCreate:
Initial scaffold of Books and Authors tables by EF convention.

2. AnnotateAuthor:
Table mapped to `Writers`, `Name` Required & MaxLength(100).

3. FluentBook:
Configured `BookConfigurations`: Title(150), Price decimal(8,2).